

Factory Method Design Pattern

Intent

Define an interface for creating an object, but let the *factory* decide which concrete class to instantiate. The caller asks for a product **by a key** (here an enum) and gets back the right implementation without ever naming the concrete class or calling `new` on it.

This implementation is the **Spring-idiomatic, self-registering** variant: every product is a Spring `@Component`, they are all injected as a `List`, and the factory indexes them into an `EnumMap`. Adding a new shape requires **zero changes** to the factory.

UML class diagram

```
<<interface>> Shape +-----+ | +draw() | | +getDesignType() |  
+-----^-----+ | implements +-----+-----+-----+ | | | +-----+  
+---+-----+ +---+-----+ | |Circle| |Rectangle| |Triangle | | +-----+ +-----+  
+-----+ | collected into | +-----+-----+-----+ | ShapeFactory |  
+-----+-----+ | - ShapeMap : EnumMap<DesignType,Shape> | +  
getShape(DesignType) : Shape | +-----+
```

The players

```
enums/DesignType the key (CIRCLE, TRIANGLE, RECTANGLE) factory/contract/Shape the product  
interface factory/contract/concret/Circle concrete products, each an @Component /Rectangle  
/Triangle factory/ShapeFactory the factory – maps DesignType → Shape
```

The code, line by line

`DesignType` — the key enum

```
public enum DesignType { CIRCLE("CIRCLE"), TRIANGLE("TRIANGLE"), RECTANGLE("RECTANGLE");  
private String name; private DesignType(String name) { this.name = name; } @JsonValue  
public String getName() { return name; } }
```

- An enum is the discriminator that tells the factory *which* product you want. Using an enum instead of a `String` gives you **compile-time safety** — you can't ask for a shape that doesn't exist, and it makes the `EnumMap` (below) possible.

- `@JsonValue` — when this enum is serialized to/from JSON (e.g. as a request field), Jackson uses `getName()` as its wire value instead of the default `name()`. Not needed by the pattern itself; it's there so the enum is API-friendly.

Shape — the product interface

```
public interface Shape { DesignType getDesignType(); void draw(); }
```

- `draw()` is the actual behavior a product offers.
- `getDesignType()` is the key idea that makes self-registration work: **each product declares its own key**. The factory doesn't need a big switch mapping classes to enums — it just asks each product "what are you?".

Circle / Rectangle / Triangle — concrete products

```
@Component public class Circle implements Shape { @Override public void draw() {
    System.out.println("Shape is circle."); } @Override public DesignType getDesignType() {
    return DesignType.CIRCLE; } }
```

- `@Component` — registers each shape as a Spring bean. This is what lets Spring discover them all and inject them as a list. Without it, the factory would have nothing to collect.
- `getDesignType()` returns *this* product's identity. `Circle` says `CIRCLE`, `Triangle` says `TRIANGLE`, and so on.

ShapeFactory — the factory itself

```
@Component public class ShapeFactory { private final EnumMap<DesignType, Shape> shapeMap;
    private final Shape defaultShape; public ShapeFactory(List<? extends Shape> shapes, Circle
    circle) { this.defaultShape = Objects.requireNonNull(circle, "default circle must not be
    null"); this.shapeMap = shapes.stream().collect(Collectors.toMap( Shape::getDesignType, //
    key = each product's own type Function.identity(), // value = the product itself (a, b) ->
    b, // merge = if duplicate key, keep the later one () -> new
    EnumMap<>(DesignType.class))); // backing map = EnumMap } public Shape getShape(final
    DesignType designType) { return shapeMap.getDefault(designType, defaultShape); } }
```

Constructor — `List<? extends Shape> shapes:`

- Spring injects **every** bean that implements `Shape` as a list. This is the crux of self-registration: the factory never mentions `Circle`, `Rectangle`, `Triangle` by name (except for the default). New implementations get picked up automatically just by being `@ComponentS`.
- `Circle circle` is injected separately to serve as the **default/fallback** product.

Building the map with a stream + `collectors.toMap`:

- `Shape::getDesignType` — the key extractor. Each shape is filed under the key *it* reports.
- `Function.identity()` — the value is the shape object itself (`x -> x`).
- `(a, b) -> b` — the **merge function**. `toMap` throws on duplicate keys unless you supply one; this says "if two shapes claim the same `DesignType`, keep the second." It prevents a startup crash if a key is accidentally duplicated.
- `() -> new EnumMap<>(DesignType.class)` — the **map supplier**. Forces the result to be an `EnumMap` instead of the default `HashMap`.

`getShape(...)`:

- `getOrDefault(designType, defaultShape)` — one clean line: look the key up, and if nothing is registered for it, hand back the default `circle` instead of `null`. The caller never has to null-check.

Why the design decisions

Why an `EnumMap` instead of a `switch` or a `HashMap`?

- **VS. `switch`** — a `switch(designType)` with a case per shape means the factory has to be **edited every time** you add a shape (Open/Closed Principle violation). The map-based approach means a new shape is discovered automatically; the factory code is closed for modification. This is the exact refactor this project standardized on ("stop using switch cases, use `EnumMap`").
- **VS. `HashMap`** — `EnumMap` is purpose-built for enum keys. Internally it is backed by a plain array indexed by the enum's `ordinal()`, so lookups are array-index-fast with no hashing and no collisions, and it uses less memory. Whenever the key is an enum, `EnumMap` is the right container.

Why inject a `List<? extends Shape>` (self-registration)?

Because it inverts the dependency. A hand-written factory *knows about* all its products (it imports and *news* them). Here the factory knows about **none** of them — it just receives whatever `Shape` beans exist and files them by their self-reported key. Adding `Hexagon`:

1. create `Hexagon` implements `Shape`, annotate `@Component`, return `DesignType.HEXAGON`;
2. add `HEXAGON` to the enum.

That's it — `ShapeFactory` is **never touched**.

Why each product reports its own `getDesignType()`?

So the factory doesn't need any external mapping table. The alternative (a central "Circle→CIRCLE" registry) duplicates knowledge and drifts out of sync. Letting the product own its key keeps the single source of truth inside the product.

Why a default shape via `getOrDefault`?

Robustness. If someone asks for a `DesignType` that has no registered implementation, returning a sensible default is friendlier than returning `null` and forcing every caller to guard against `NullPointerException`. `Objects.requireNonNull` in the constructor guarantees the default itself is never null.

Why `final` fields?

`shapeMap` and `defaultShape` are set once in the constructor and never change. Marking them `final` makes the factory **immutable after construction** and therefore inherently thread-safe to read — no synchronization needed for concurrent `getShape()` calls.

Execution flow (as run from `main`)

```
DesignPatternsApplication.main ■ ■■■ SpringApplication.run(...) Spring starts, discovers
@Component ■ ■■■ constructs Circle, Rectangle, Triangle beans ■ ■■■ constructs
ShapeFactory, injecting List[Circle,Rectangle,Triangle] + Circle ■ ■■■ builds EnumMap {
CIRCLE→Circle, TRIANGLE→Triangle, RECTANGLE→Rectangle } ■ ■■■
context.getBean(ShapeFactory.class) fetch the factory from the context ■ ■■■
shapeFactory.getShape(DesignType.TRIANGLE) ■■■ shapeMap.getOrDefault(TRIANGLE, circle) →
Triangle bean ■■■ .draw() → prints "Shape is triangle"
```

Factory Method vs. Abstract Factory (so you don't confuse them)

- **Factory Method (this module)** — produces **one** product from a family, chosen by a key. One axis: "which shape?"
- **Abstract Factory** (the sibling module) — produces a **matched set** of related products (a chair *and* a table that belong together). The factory itself *is* the choice; you don't pass a key per product.

If you ever find yourself passing a "type" argument into every `create...()` call, you're doing Factory Method — which is correct here, and is exactly what the Abstract Factory module was fixed **not** to do.

Note: `SpringApplication.run(...)` boots the container so the `@Component` products can be discovered and injected. The pattern itself doesn't require Spring — you could build the

`EnumMap` by hand — but Spring's component scanning is what makes the self-registration automatic.